

SORTING DI UN MULTISET DI STRINGHE

Martedì 28 Aprile 2026

1



ORDINARE N ITEMS. QUALE LOWER BOUND?

- Per ordinare un multiset di n items atomici (che hanno una lunghezza fissata, per esempio numeri reali, interi, caratteri,...) e per i quali è definita una relazione d'ordine totale è possibile usare algoritmi basati su confronto (il confronto tra due elementi è realizzato in tempo costante). $\Omega(n \log n)$ è un lower bound ben noto.
- Esistono differenti algoritmi (per esempio quicksort, mergesort, heapsort...) che ordinano un multiset di n elementi usando $O(n \log n)$ confronti (worst o average case)
- Osservazione 1: Se il tipo di operazione è diverso è possibile trovare algoritmi più efficienti.
- Osservazione 2: Se si ordinano elementi non-atomici (a lunghezza variabile), come le stringhe questo lower bound non è adeguato.

COUNTING SORT

- E' un algoritmo che ordina n interi in tempo $O(n)$.
- Il lower bound $\Omega(n \log n)$ per il sorting non si applica ad algoritmi che usano operazioni più forti del confronto. Qui si usa il conteggio delle occorrenze.

Counting Sort

Input: Array $R = [k_1, \dots, k_n]$ di interi (insieme con ordine totale) nel range $[0, \sigma)$

Output: array $J[0, n)$ contenente gli elementi di R ordinati in modo non-decrescente

```
(1) for  $i \leftarrow 0$  to  $\sigma - 1$  do  $C[i] \leftarrow 0$ 
(2) for  $i \leftarrow 1$  to  $n$  do  $C[k_i] \leftarrow C[k_i] + 1$ 
(3)  $sum \leftarrow 0$ 
(4) for  $i \leftarrow 0$  to  $\sigma - 1$  do // cumulative sums
(5)    $tmp \leftarrow C[i]; C[i] \leftarrow sum; sum \leftarrow sum + tmp$ 
(6) for  $i \leftarrow 1$  to  $n$  do // distribute
(7)    $J[C[k_i]] \leftarrow k_i; C[k_i] \leftarrow C[k_i] + 1$ 
(8) return  $J$ 
```

- l' array R è analizzato da sinistra a destra
- E' un algoritmo stabile. La complessità di tempo e spazio è $O(n + \sigma)$



ESEMPIO — COUNTING SORT

```
(1) for  $i \leftarrow 0$  to  $\sigma - 1$  do  $C[i] \leftarrow 0$ 
(2) for  $i \leftarrow 1$  to  $n$  do  $C[k_i] \leftarrow C[k_i] + 1$ 
(3)  $sum \leftarrow 0$ 
(4) for  $i \leftarrow 0$  to  $\sigma - 1$  do // cumulative sums
(5)    $tmp \leftarrow C[i]; C[i] \leftarrow sum; sum \leftarrow sum + tmp$ 
(6) for  $i \leftarrow 1$  to  $n$  do // distribute
(7)    $J[C[k_i]] \leftarrow k_i; C[k_i] \leftarrow C[k_i] + 1$ 
(8) return  $J$ 
```

Conta le
occorrenze
(line 2)

a	2	0
b	3	2
c	1	5
d	2	6
e	1	8
f	3	9

N. Elementi
< d
Rappresenta
la posizione
più a sinistra
contenente d

d	0	a
a	1	a
c	2	b
f	3	b
f	4	b
b	5	c
d	6	d
b	7	d
f	8	e
b	9	f
e	10	f
a	11	f



ORDINARE UN MULTISSET DI STRINGHE

- Problema: Dato un multiset di n stringhe, ordinarle in modo non-decrescente.
- Se usiamo un algoritmo basato su confronti (come quicksort, mergesort, heapsort) per ordinare stringhe, è facile capire che il numero di confronti può essere maggiore di $O(n \log n)$ nel caso peggiore.
- Confrontare due stringhe A e B richiede almeno $|lcp(A,B)|$ confronti (longest common prefix) e $lcp(A,B)$ può essere arbitrariamente grande in generale.
- Se usiamo strategie appropriate che confrontano stringhe vicine l'una all'altra in ordine lessicografico (hanno lunghi prefissi comuni), possiamo trarne vantaggio.

LCP ARRAY

- Sia $R = \{S_1, S_2, \dots, S_n\}$ un multiset di stringhe. Assumiamo che $S_1 \leq S_2 \leq \dots \leq S_n$.
- Il **LCP array** $LCP_R[1..n]$ è così definito:
 $LCP_R[1] = 0$ e per $i \in [2..n]$ $LCP_R[i] = |\text{lcp}(S_i, S_{i-1})|$.

- Inoltre, $\sum LCP(R) = \sum_{i \in [1..n]} LCP_R[i]$

- Ex: $R = \{\text{ali}, \text{alice}, \text{anna}, \text{eliana}, \text{elias}, \text{elisa}\}$

LCP_R

0
3
1
0
4
3

$$\sum LCP(R) = 11$$

LOWER BOUND

- **TEOREMA:** Sia $R = \{S_1, S_2, \dots, S_n\}$ un insieme di n stringhe. Ordinare R in ordine lessicografico mediante un algoritmo basato su confronto di simboli, richiede $\Omega(\sum \text{LCP}(R) + n \log n)$ confronti.
- **Idea della prova:** Se le stringhe fossero nell'ordine corretto, e dovessimo solo verificare che lo sono, ci servirebbero $\Omega(\sum \text{LCP}(R))$ confronti. D'altra parte per averle nell'ordine corretto servono $\Omega(n \log n)$ confronti.



STRING QUICKSORT (3-WAY QUICKSORT)

- Introdotto da Bentley e Sedgewick nel 1997

Fast Algorithms for Sorting and Searching Strings

Jon L. Bentley*

Robert Sedgewick#

Abstract

We present theoretical algorithms for sorting and searching multikey data, and derive from them practical C implementations for applications in which keys are character strings. The sorting algorithm blends Quicksort and radix sort; it is competitive with the best known C sort codes. The searching algorithm blends tries and binary

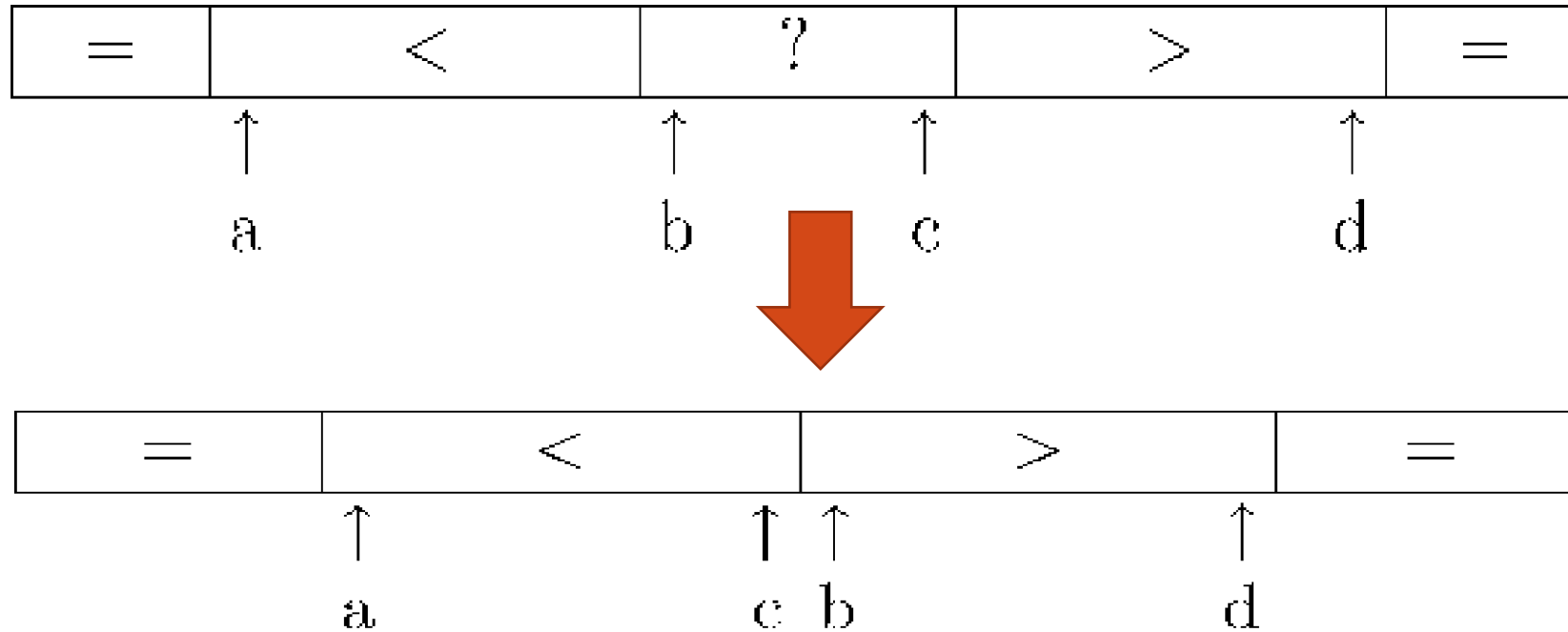
that is competitive with the most efficient string sorting programs known. The second program is a symbol table implementation that is faster than hashing, which is commonly regarded as the fastest symbol table implementation. The symbol table implementation is much more space-efficient than multiway trees, and supports more advanced searches.

3-WAY QUICKSORT VS QUICKSORT

- Quicksort è un algoritmo basato sul paradigma divide-and-conquer. Per ordinare un array R , scegliamo un elemento pivot, scambiamo gli elementi portando quelli minori del pivot da una parte e quelli maggiori dall'altra, e applicando ricorsivamente l'algoritmo ai due sotto-array.
- In questo modo, potremmo avere due sottoinsiemi $R_{\leq}$ e $R_{\geq}$, entrambi contenenti elementi uguali al pivot.
- L'idea della partizione ternaria ($R_{<}, R_{=}, R_{>}$) era nata un po' di tempo prima ma senza grossi successi:
 - Binary quicksort è leggermente più veloce in pratica per ordinare insiemi
 - Ternary quicksort può essere più veloce per ordinare multiset con chiavi duplicate.
- Dijkstra nel 1976 lo rese popolare come "**Il problema della bandiera nazionale olandese**": dobbiamo ordinare una sequenza di sassolini rossi, bianchi e blu per farli apparire nel loro ordine sull'insegna dell'Olanda. Questo corrisponde al partizionamento Quicksort quando gli elementi minori sono colorati di rosso, gli elementi uguali sono bianchi e gli elementi maggiori sono blu. L'algoritmo ternario di Dijkstra richiede tempo lineare (guarda ogni elemento esattamente una volta), ma 'il codice per implementarlo ha un fattore costante significativamente più grande del codice di partizionamento binario'.

ESEMPIO DI PARTIZIONE TERNARIA

- Split-end partition (usata da Bentley e Sedgwick)



- Alla fine, gli elementi uguali dalle due estremità sono «swappati» al centro.

STRING QUICKSORT (3-WAY QUICKSORT)

StringQuicksort($\mathcal{R}, \ell$)

INPUT: (Multi)set $\mathcal{R}$ ordine arbitrario.

OUTPUT: $\mathcal{R}$ in ordine non decrescente

- (1) if $|\mathcal{R}| \leq 1$ then return $\mathcal{R}$
- (2) $\mathcal{R}_\perp \leftarrow \{S \in \mathcal{R} \mid |S| = \ell\}$; $\mathcal{R} \leftarrow \mathcal{R} \setminus \mathcal{R}_\perp$
- (3) select pivot $X \in \mathcal{R}$
- (4) $\mathcal{R}_< \leftarrow \{S \in \mathcal{R} \mid S[\ell] < X[\ell]\}$
- (5) $\mathcal{R}_= \leftarrow \{S \in \mathcal{R} \mid S[\ell] = X[\ell]\}$
- (6) $\mathcal{R}_> \leftarrow \{S \in \mathcal{R} \mid S[\ell] > X[\ell]\}$
- (7) $\mathcal{R}_< \leftarrow \text{StringQuicksort}(\mathcal{R}_<, \ell)$
- (8) $\mathcal{R}_= \leftarrow \text{StringQuicksort}(\mathcal{R}_=, \ell + 1)$
- (9) $\mathcal{R}_> \leftarrow \text{StringQuicksort}(\mathcal{R}_>, \ell)$
- (10) return $\mathcal{R}_\perp \cdot \mathcal{R}_< \cdot \mathcal{R}_= \cdot \mathcal{R}_>$

Nella chiamata iniziale, $\ell=0$.



ELEMENTO PIVOT

- L'elemento pivot può essere scelto in vari modi, per es. computando la mediana tra alcuni elementi o scegliendo a caso uno degli elementi.
- Il partizionamento dell'array in base all'elemento pivot può essere implementato in vari modi:
 - *E.Kim, K. Park, Improving multikey Quicksort for sorting strings with many equal elements, Information Processing Letters, 2009.*



ESEMPIO...

partitioning item

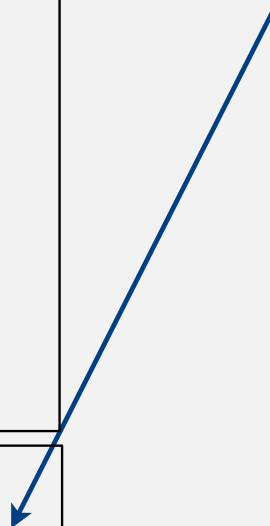
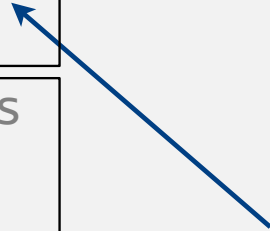


use first character to
partition into
"less", "equal", and "greater"
subarrays

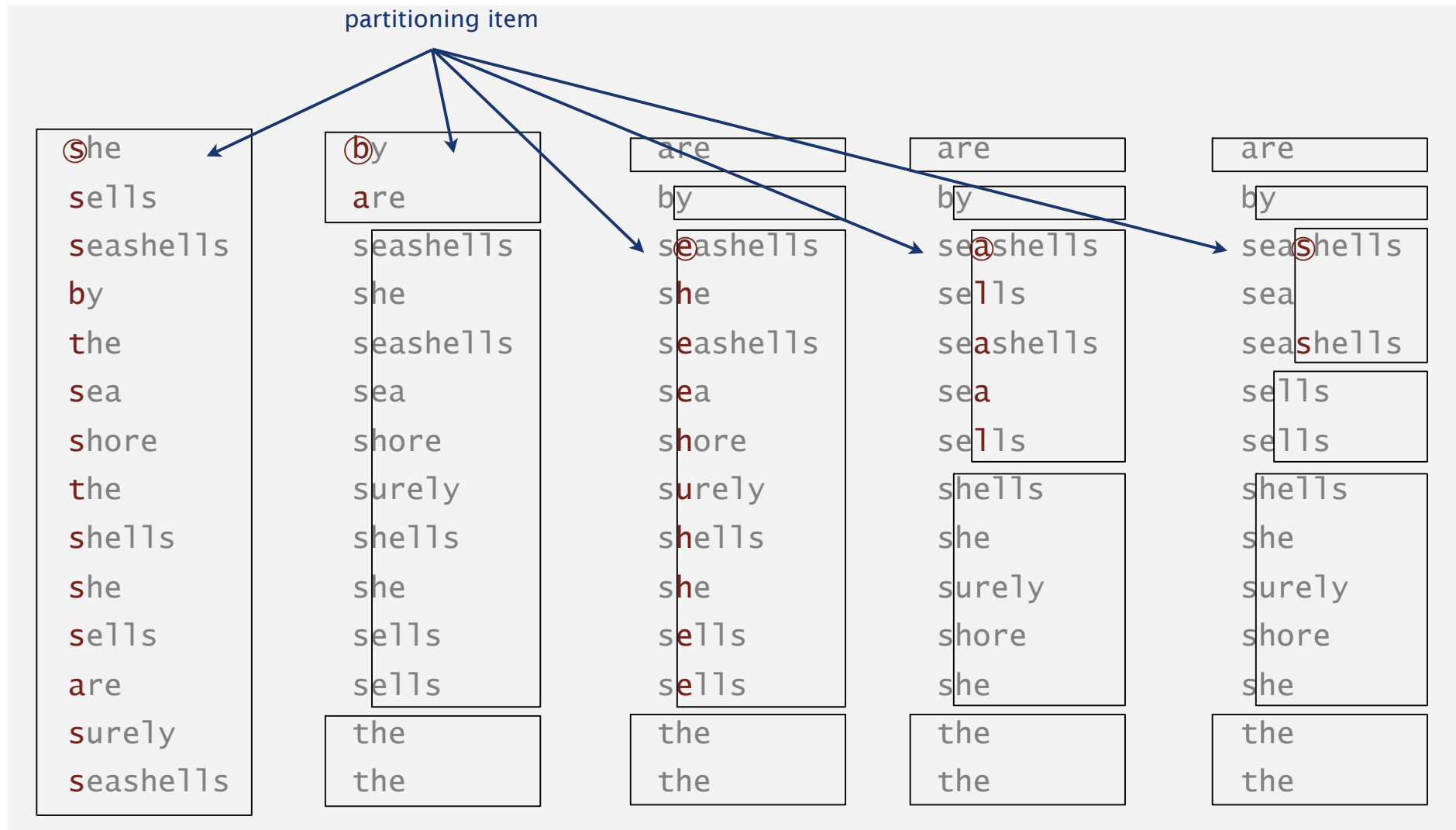
She
sells
seashells
by
the
sea
shore
the
shells
she
sells
are
surely
seashells

by
are
seashells
she
seashells
sea
shore
surely
shells
she
sells
sells
the
the

recursively sort subarrays,
excluding first character
for middle subarray



LE PRIME CHIAMATE RICORSIVE...



STRING QUICKSORT: COMPLESSITÀ

TEOREMA: StringQuicksort ordina un insieme R di n stringhe in $O(\sum LCP(R) + n \log n)$ in media. Tale bound diventa anche un worst-case adottando una strategia lineare per selezionare l'element pivot come mediana di R .

Idea della prova: La complessità di tempo è dominata dai confronti dei simboli nelle linee (4)–(6).

Se $S[\ell] = X[\ell]$, la stringa S è posizionata nell'insieme $R_{\leq}$. La chiamata ricorsiva su $R_{\leq}$ incrementa la lunghezza del prefisso comune a $\ell + 1$. Quindi $S[\ell]$ non sarà coinvolto in futuri confronti, quindi abbiamo 1 confronto.

Soltanto $\text{lcp}(S, R \setminus \{S\})$ simboli in S possono essere coinvolti in questi confronti. Quindi, il numero totale di confronti è al più $\sum \text{lcp}(R) = \Theta(\sum LCP(R))$.

Se $S[\ell] \neq X[\ell]$, la stringa S è posizionata in $R_{<}$ o $R_{>}$. La taglia di ciascun insieme è al più $|R|/2$ assumendo una scelta ottimale del pivot X (per esempio la mediana di R).

Ogni confronto caricato su S dimezza la taglia dell'insieme che lo contiene, quindi il totale di tempo accumulato è al più $\log n$. Quindi il numero totale di confronti risulterà al più $O(n \log n)$.

OSSERVAZIONI:

- Tale algoritmo evita di riconfrontare lunghi prefissi comuni (che invece rappresenta un limite per il quicksort).
- Il lower bound $\Omega(\sum LCP(R) + n \log n)$ non si applica all'ordinamento di stringhe che non usano il confronto tra simboli.



ESERCIZIO

- Implementare lo String Quicksort.
- E' un algoritmo stabile?

RADIX SORT

- Radix sort è un algoritmo sviluppato per ordinare interi grandi, ma tali interi sono trattati come stringhe di cifre, quindi è in effetti un algoritmo per ordinare stringhe.
- Ci sono due tipi di radix sort:
 - MSD radix sort : inizia l'ordinamento dall'inizio delle stringhe (most significant digit)
 - LSD radix sort: inizia l'ordinamento dalla fine delle stringhe (least significant digit)



LSD

- LSDRadixSort(R)

INPUT: (Multi)set $R = \{S_1, S_2, \dots, S_n\}$ di stringhe di lunghezza m sull'alfabeto $[0..\sigma)$.

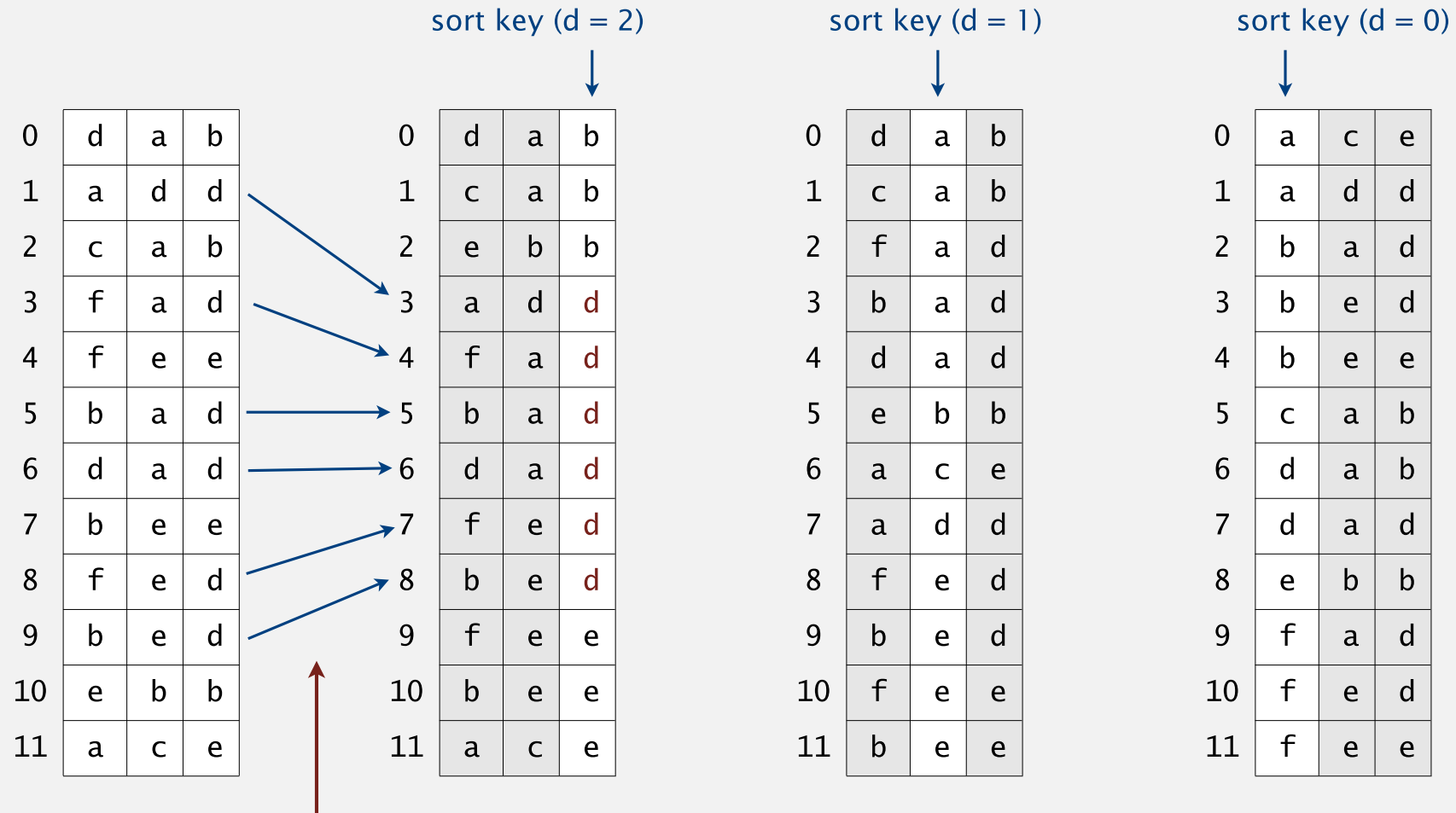
OUTPUT: R in ordine lessicografico

```
(1) for  $\ell \leftarrow m - 1$  to 0 do CountingSort( $\mathcal{R}, \ell$ )  
(2) return  $\mathcal{R}$ 
```

- CountingSort(R, ℓ) ordina le stringhe in R in base ai simboli in posizione ℓ usando il counting sort . La complessità di tempo è $O(|R| + \sigma)$
- La stabilità del counting sort è essenziale.

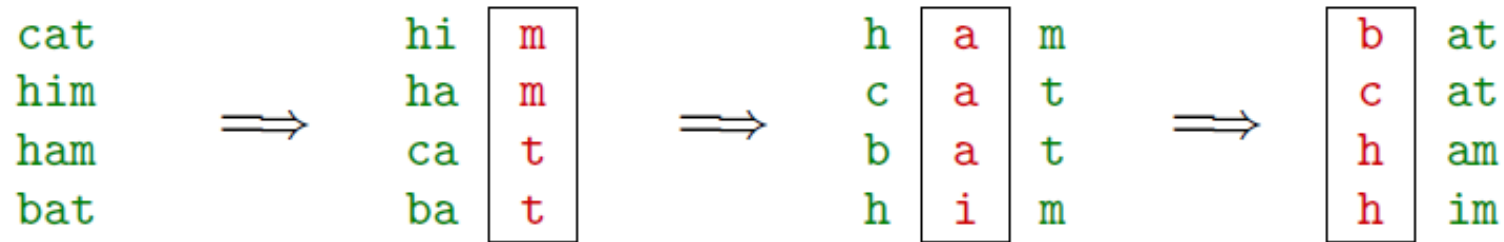


ESEMPIO



LSD

- Altro ESEMPIO:



- E' facile verificare che dopo k passi le stringhe sono ordinate per il suffisso di lunghezza k .
- TEOREMA: LSD radix sort ordina un insieme R di stringhe su un alfabeto $[0..\sigma)$ in $O(||R|| + m \sigma)$ tempo, dove $||R||$ è la lunghezza totale delle stringhe in R e m è la lunghezza delle stringhe.
DIM: la prova segue dalla complessità del counting sort.
- La debolezza del LSD-radix sort consiste nel fatto di usare un tempo $\Omega(||R||)$ anche se $\Sigma\text{LCP}(R)$ è molto minore di $||R||$. Funziona comunque bene per stringhe e interi corti.



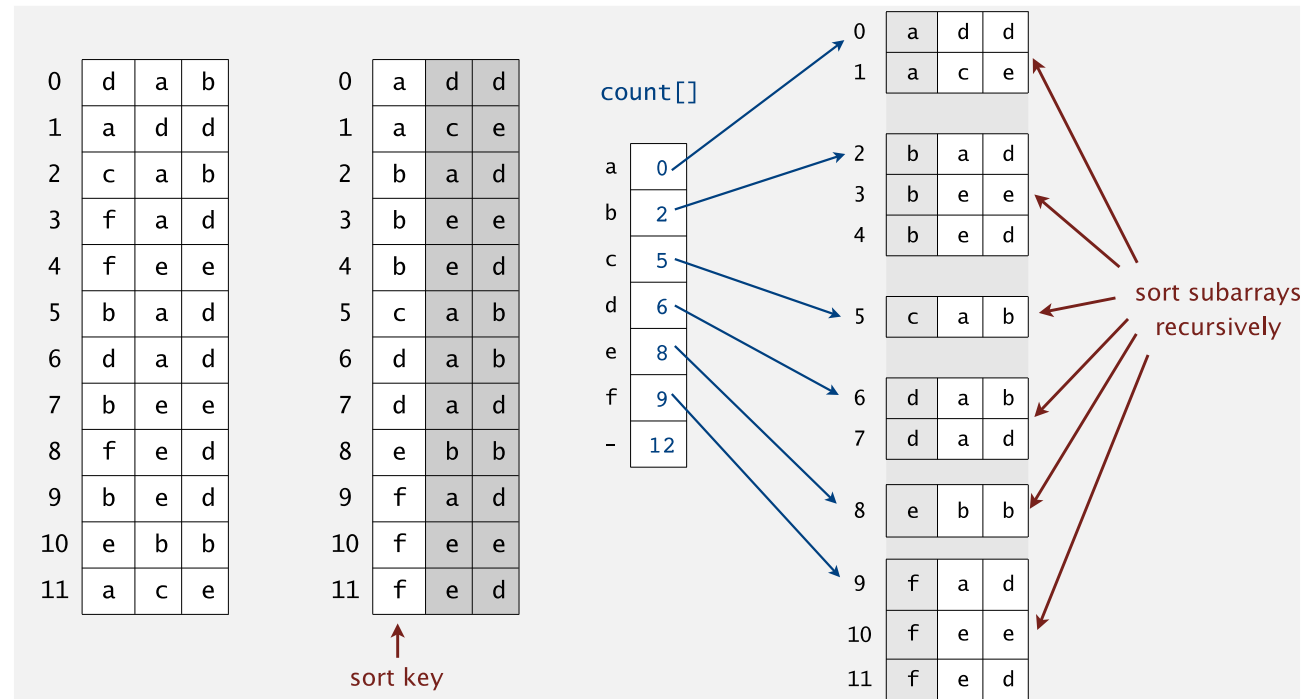
SI PUÒ APPLICARE LSD DA SINISTRA A DESTRA?

				sort key (d = 0)								sort key (d = 1)								sort key (d = 2)			
				↓								↓								↓			
0	d	a	b		0	a	d	d		0	b	a	d		0	c	a	b		0	c	a	b
1	a	d	d		1	a	c	e		1	c	a	b		1	d	a	b		1	d	a	b
2	c	a	b		2	b	a	d		2	d	a	b		2	e	b	b		2	e	b	b
3	f	a	d		3	b	e	e		3	d	a	d		3	b	a	d		3	b	a	d
4	f	e	e		4	b	e	d		4	f	a	d		4	d	a	d		4	d	a	d
5	b	a	d		5	c	a	b		5	e	b	b		5	f	a	d		5	f	a	d
6	d	a	d		6	d	a	b		6	a	c	e		6	a	d	d		6	a	d	d
7	b	e	e		7	d	a	d		7	a	d	d		7	b	e	d		7	b	e	d
8	f	e	d		8	e	b	b		8	b	e	e		8	f	e	d		8	f	e	d
9	b	e	d		9	f	a	d		9	b	e	d		9	a	c	e		9	a	c	e
10	e	b	b		10	f	e	e		10	f	e	e		10	b	e	e		10	b	e	e
11	a	c	e		11	f	e	d		11	f	e	d		11	f	e	e		11	f	e	e

not sorted!

MSD

- Partiziona il multiset in σ pezzi in accordo al primo carattere
- Ordina ricorsivamente tutte le stringhe che cominciano con ciascuno dei caratteri.



MSD

- **MSDRadixSort($\mathcal{R}, \ell$)**
INPUT: (Multi)set $\mathcal{R} = \{S_1, S_2, \dots, S_n\}$ di stringhe sull'alfabeto $[0..\sigma)$ e lunghezza ℓ del prefisso comune
OUTPUT: $\mathcal{R}$ in ordine lessicografico
 - (1) if $|\mathcal{R}| < \sigma$ then return StringQuicksort($\mathcal{R}, \ell$)
 - (2) $\mathcal{R}_\perp \leftarrow \{S \in \mathcal{R} \mid |S| = \ell\}$; $\mathcal{R} \leftarrow \mathcal{R} \setminus \mathcal{R}_\perp$
 - (3) $(\mathcal{R}_0, \mathcal{R}_1, \dots, \mathcal{R}_{\sigma-1}) \leftarrow \text{CountingSort}(\mathcal{R}, \ell)$
 - (4) for $i \leftarrow 0$ to $\sigma - 1$ do $\mathcal{R}_i \leftarrow \text{MSDRadixSort}(\mathcal{R}_i, \ell + 1)$
 - (5) return $\mathcal{R}_\perp \cdot \mathcal{R}_0 \cdot \mathcal{R}_1 \cdots \mathcal{R}_{\sigma-1}$
- **CountingSort($\mathcal{R}, \ell$)** ritorna anche la partizione basata sul simbolo in posizione ℓ . La complessità di tempo è $O(|\mathcal{R}| + \sigma)$
- Potrebbe essere prodotto un gran numero di insiemi molto piccoli. Counting sort necessita $\Omega(\sigma)$ tempo indipendentemente dalla dimensione dell'insieme. Per tale motivo di usa lo StringQuicksort per insiemi piccoli.
- **TEOREMA:** MSD radix sort ordina un insieme $\mathcal{R}$ di n stringhe sull'alfabeto $[0..\sigma)$ in tempo $O(\text{LCP}(\mathcal{R}) + n \log \sigma)$.
- Esiste una variante con complessità $O(\Sigma \text{LCP}(\mathcal{R}) + n + \sigma)$. Osserviamo che il lower bound per algoritmi che non sono basati su confronto è $\Omega(\Sigma \text{LCP}(\mathcal{R}) + n)$



TRACCIA DELLE CHIAMATE RICORSIVE...



MSD – NUMERO DEI CARATTERI ESAMINATI

- In pratica, MSD radix sort è molto veloce. Il numero dei caratteri esaminati dipende dalle chiavi. Può anche essere sublineare

Random (sublinear)	Non-random with duplicates (nearly linear)	Worst case (linear)
1EI0402	are	1DNB377
1HYL490	by	1DNB377
1ROZ572	sea	1DNB377
2HXE734	seashells	1DNB377
2IYE230	seashells	1DNB377
2XOR846	sells	1DNB377
3CDB573	sells	1DNB377
3CVP720	she	1DNB377
3IGJ319	she	1DNB377
3KNA382	shells	1DNB377
3TAV879	shore	1DNB377
4CQP781	surely	1DNB377
4QGI284	the	1DNB377
4YHV229	the	1DNB377

3-WAY QUICKSORT VS MSD

MSD

- Ha molti accessi random alla memoria (cache inefficient)
- Extra spazio per contatori e variabili temporanee

3-way Quicksort

- E' cache-friendly
- E' in-place